第 1 次作业

第 1 章习题

1.5 计算序列各程序段的时间复杂度。

（1）for (i=0; i<n; i++)

for (j=i; j<n;j++) x++;

【解】$O(n^2)$

i 值	x++执行次数
0	n-1
1	n-2
2	n-3
...	...
n-2	1
n-1	0

x++执行总次数：1+2+3+..+(n-1)=n(n-1)/2

时间复杂度：$O(n^2)$

（2）i=n;

while (i>1) i= i/2;

【解】$O(\log_2 n)$。假定 i=i/2 运行了 k 次，则 k 与 i 值之间的关系如下表：

k 值	1	2	3	...	k
i 值	i=n/2	$i=n/2^2$	$i=n/2^3$	...	$i=n/2^k$

循环结束时，i 接近 1，不妨设为 1，则有 $1= n/2^k$ ，可得循环次数 $k=\log_2 n$。

时间复杂度：$O(\log_2 n)$

（3）for (i=1; i<=n; i++)

for (j=1; j<=n; j++)

for (k=1; k<=n; k++)

x++;

【解】$O(n^3)$

（4）for (i=1; i<n; i++)

for (j=1; j<n; j++) x++;

for (k=1; k<n;k++) x++;

【解】$O(n^2)$

（5）for (i=1; i<n; i++)

```
{   j=i;
    while (j<n) j*=2;
}
```

【解】O(n)

i 值与 j*=2 近似执行次数对应表

i 值	j*=2 近似执行次数
1	$\log_2 n$
2	$\log_2 n- \log_2 2$
3	$\log_2 n- \log_2 3$
...	..
n-1	$\log_2 n- \log_2 (n-1)$

j*=2 近似执行总次数：$n\log_2 n-(\log_2 2+ \log_2 3+..+ \log_2 (n-1)+ \log_2 n)=n\log_2 n-\log_2 n!$。

根据 sterling 公式，当 n 很大时，$n!\approx\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$，代入得，

$$n\log_2 n-\log_2 n!=n\log_2 n-\frac{1}{2}\log_2 2\pi n-n\log_2 n+n\log_2 e$$

所以，时间复杂度：O(n)。

第 2 章习题

2.7 假设顺序表 L 中的元素按从小到大的次序排列，设计算法以删除表中重复的元素，并要求时间尽可能少。要求：

（1）对顺序表（1,1,2,2,2,3,4,5,5,5,6,6,7,7,8,8,8,9）模拟执行本算法，并统计移动元素的次数。

（2）分析算法的时间性能。

【解】

【分析】

将元素分成两个部分：已经处理元素和待处理元素。已经处理部分返回 L 中，用下标 i 指示最后一个元素，初始化 i=0。待处理部分用下标 j 指示第一个元素，初始化 j=1.

左边下标小于 i 的元素已经处理好重复，等于 i 是当前正在处理的元素，将 data[i]与 data[j]

进行比较，会出现下列情况：

① data[i]==data[j]，说明 j 指示的是 i 的重复元素，继续处理 j 的下一个元素，即执行 j++。

② data[i]<data[j]， 说明 j 指示元素与 i 的元素不同，如果 i+1!=j，将 j 元素复制到 i+1，即：L.data[i+1]=L.data[j]，再执行 j++，i++；若 i+1==j，说明 j 是 i 的直接后继，无需复制，直接执行 i++，j++。

循环执行上述操作，直到表尾。

修改 L 的长度为 i+1。

【算法描述】

```
void DeleteRepeatData(seqList & L)
{
    int i,j;   //分别指向已处理部分最后元素和未处理部分第一个元素，皆为数组下标

    if(L.listLen<2)
        return;    //少于 2 个元素，直接退出
    i=0;   //初始化 i 指向第一个元素
    j=1;   //j 指向第二个元素
    while(j<L.listLen)
    {
        if(L.data[i]==L.data[j])   //j 为重复元素，j 后移
            j++;
        else   //因为 L 递增，所以剩下情况即 L.data[i]<L.data[j]，j 为目标元素
        {
                //如果 j==i+1，说明 j 紧随 i，无需移动元素，直接 i++、j++即可
            if((i+1)!=j)
                L.data[i+1]=L.data[j];   //j 元素复制到 i+1

            i++;   //无论那种情况，都需要同时后移 i、j
            j++;
        }
    }
    L.listLen=i+1;   //修改表的实际长度
}
```

【算法分析】时间复杂度 O(n)。上例中只需移动 8 个元素。如果有 n 个不同元素，则最多移动元素 n-1 次。

2.9 递增有序顺序表 A、B 分别表示一个集合，设计算法求解 A=A－B，并分析其时间性能。

【解】【分析】

审题：A=A-B，即要求要利用 A 表的空间保存差集元素，而不是创建一个新表。

设置两个指针 ia、ib 分别指向 A、B 表当前处理的元素；

设置一个指针 i 指示已经求取的差集元素在 A 的表中的最后元素位置；

比较 A、B 表当前元素，会出现以下三种情况

(1) A.data[ia]==B.data[ib]，则 ia 或 ib 是交集元素，不是 A-B 中元素，直接跳过，修改指针：ia++，ib++。

(2) A.data[ia]>B.data[ib]，ia 指示的元素可能在 B 表 ib 指示的元素后面，ia 不动，移动 ib，即 ib++。

(3) A.data[ia]<B.data[ib]，ia 指示的元素不可能在 B 中出现，故比为 A-B 中元素。需要的话迁移到目标位置，即 (i+1)!=ia 时，执行 A.data[i+1]=A.data[ia]。若(i+1)==ia，ia 即为目标位置，无需复制迁移。迁移完成，移动指示器：i++，ia++。

重复以上过程，直至 A、B 中至少一个表结束。

还有一种情况：B 表已结束，但 A 表尚未结束，说明 A 剩下元素不在 B 中，全为 A-B 中元素，全部迁移到目标位置。

最后，修改 A 表长度为 i+1。

【算法描述】

```
void SetSubtraction(seqList & A, seqList & B)
{
    int i=-1;   //指示 A 中已经处理的最后元素
    int ia=0, ib=0;   //指示 A、B 中，当前待处理的元素，初始指向第一个元素
    while( ia<A.listLen && ib<B.listLen )
    {
        if(A.data[ia]==B.data[ib])     //非 A-B 中元素，ia、ib 同时后移
        {
            ia++;
            ib++;
        }
        else if(A.data[ia]>B.data[ib])
            ib++;     //此时，ia 指示元素可能在 B 中 ib 指示的元素后面，移动 ib。
        else   //此为，A.data[ia]<B.data[ib]，因为递增性，ia 指示的元素不可能在 B 中。
        {      //所以 ia 指示元素必在 A-B 中。
            //如果(i+1)==ia，说明 ia 元素不需迁移位置，直接为 A-B 中元素
            if(i+1!=ia)   //(i+1!=ia)，需要将 ia 指示元素迁移到目标位置 i+1
                A.data[i+1]=A.data[ia];

            i++;   //A-B 集合最后元素指示器后移
            ia++; //A 的指示器后移
        }
    }
        //处理 B 已经，A 尚未结束情况，A 中剩下部分元素全部为 A-B 元素
    while(ia<A.listLen)
```

```
    {
        if(i+1!=ia)
            A.data[i+1]=A.data[ia];

        i++;   //A-B 集合最后元素指示器后移
        ia++; //A 的指示器后移
    }
    A.listLen=i+1;   //更新表 A 的长度，使等于|A-B|
}
```

【算法分析】时间性能 O(|A| + |B|)

【思考问题】A、B 两表谁先结束？

下面是本题的另一种解法，因为用到 A 表中交集元素的删除，所以效率较差。

【算法描述-1】

```
void SetSubtraction1(seqList & A, seqList & B)
{
    int ia=0, ib=0;   //ia、ib 指示 A、B 表中当前元素。初始指向第一个元素。
    while(ia<A.listLen && ib<B.listLen)
    {
        if(A.data[ia]<B.data[ib])   //元素不在 B 中，移动 A 到下一个元素
            ia++;
        else if(A.data[ia]>B.data[ib])   //A 元素可能在 B 中，移动 B 到下一个元素
        {
            ib++;
            if(ib>=B.listLen)
                break;
        }
        else   //A.data[ia]==B.data[ib]，删除 A 中元素，同时移动指针
        {
            listDelete(&A, ia+1);
            //ia++;   不能增加 ia，因为 A 已经往前移动一个元素
            ib++;
            if(ib>=B.listLen)
                break;
        }
    }
}
```

【算法分析】时间性能 O(|A| + |B|)